```
    ➥ ValidationContext validationContext)                          ❶
    {
        var otherPropertyInfo =
        ➥ validationContext.ObjectType.GetRuntime
        ➥ Property(OtherProperty);                                  ❸
        if (otherPropertyInfo == null)                               ❸
        {                                                            ❸
            return new ValidationResult(                             ❸
            ➥ "You must specify another property to compare to");   ❸
        }                                                            ❸
        var otherValue =                                             ❸
        ➥ otherPropertyInfo.GetValue(validationContext.             ❸
        ➥ ObjectInstance, null);                                    ❸
        if (!string.IsNullOrWhiteSpace
        ➥ (value?.ToString()) &&                                    ❹
        ➥ !string.IsNullOrWhiteSpace(otherValue?.ToString()) &&     ❹

        ➥ value.ToString().ToLower().First() !=                     ❹
        ➥ otherValue.ToString().ToLower().First())                  ❹
        {
            return new ValidationResult(ErrorMessage
            ?? $"The first letters of                                ❺
            ➥ {validationContext.DisplayName}                       ❺
            ➥ and {otherPropertyInfo.Name} must match");            ❺
        }                                                            ❺
        return ValidationResult.Success;                             ❺
    }
}
```

❶ Derive the class from ValidationAttribute, and override the IsValid method.

❷ Add a public property representing the name of the form property to compare to.

❸ Use the ValidationContext to get a reference to the other property.

❹ Implement the validation logic.

❺ Return a ValidationResult in the event of failure or ValidationResult.Success if validation succeeded.

The validation logic within this custom attribute is virtually identical to the page handler. You use the `ValidationContext` to access the other property's value to compare to. Its `ObjectType` property represents the object being validated, which is the `InputModel` in this case. If the other